

CO3-AT1

NAME :B.Harsha Vardhan

Reg_no :192425201

SUB CODE:MLA0302

SUB NAME: Reinforcement Learning

Exp 1 :.Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making.

CODE :

```
import random

states = ["Low Battery", "Normal Battery", "High Battery"]
actions = ["Move Left", "Move Right", "Stay"]
total_reward = 0

episodes = int(input("Enter Number of Episodes: "))
print("\nReinforcement Learning Simulation\n")

for i in range(episodes):
    state = random.choice(states)
    action = random.choice(actions)
    if state == "Low Battery" and action == "Stay":
        reward = 10
    elif state == "Normal Battery" and action == "Move Right":
        reward = 8
    elif state == "High Battery" and action == "Move Left":
        reward = 6
    else:
        reward = -5
    total_reward += reward
    print("Episode:", i + 1)
    print("State :", state)
    print("Action:", action)
```

```

print("Reward:", reward)

print("Cumulative Reward:", total_reward)

print()

print("-----")

print("Training Completed")

print("-----")

print("Total Cumulative Reward:", total_reward)

if total_reward > 0:

    print("Agent Learned Good Decisions")

else:

    print("Agent Needs More Training")

```

OUTPUT :



```

IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
RESTART: D:/MLA0302/CO3 AT1 Q1.py ==
Enter Number of Episodes: 5
Reinforcement Learning Simulation
Episode: 1
State : High Battery
Action: Stay
Reward: -5
Cumulative Reward: -5
Episode: 2
State : High Battery
Action: Stay
Reward: -5
Cumulative Reward: -10
Episode: 3
State : High Battery
Action: Move Left
Reward: 6
Cumulative Reward: -4
Episode: 4
State : Low Battery
Action: Move Left
Reward: -5
Cumulative Reward: -9
Episode: 5
State : High Battery
Action: Stay
Reward: -5
Cumulative Reward: -14
-----
Training Completed
-----
Total Cumulative Reward: -14
Agent Needs More Training
>>>

```

EXP 2: Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor.

CODE :

```

import random

print("Markov Decision Process (MDP) Simulation\n")

states = int(input("Enter Number of States: "))

actions = int(input("Enter Number of Actions: "))

```

```

gamma = float(input("Enter Discount Factor (0 to 1): "))
current_state = int(input("Enter Starting State (0 to " + str(states-1) + "): "))
total_reward = 0
for i in range(5):
    print("\nStep", i + 1)
    action = int(input("Choose Action (0 to " + str(actions-1) + "): "))
    transition_probability = round(random.uniform(0.5, 1.0), 2)
    next_state = random.randint(0, states - 1)
    reward = random.randint(-5, 10)
    discounted_reward = reward * gamma
    total_reward += discounted_reward
    print("Current State :", current_state)
    print("Action :", action)
    print("Transition Probability :", transition_probability)
    print("Next State :", next_state)
    print("Reward :", reward)
    print("Discounted Reward :", round(discounted_reward,2))
    current_state = next_state
print("\n-----")
print("MDP Simulation Completed")
print("-----")
print("Total Discounted Reward :", round(total_reward,2))

```

OUTPUT :

```

Step 3
Choose Action (0 to 4): 1
Current State : 4
Action : 1
Transition Probability : 0.58
Next State : 2
Reward : -2
Discounted Reward : -2.0

Step 4
Choose Action (0 to 4): 1
Current State : 2
Action : 1
Transition Probability : 0.61
Next State : 2
Reward : 1
Discounted Reward : 1.0

Step 5
Choose Action (0 to 4): 0
Current State : 2
Action : 0
Transition Probability : 0.71
Next State : 0
Reward : 6
Discounted Reward : 6.0

-----
MDP Simulation Completed
-----
Total Discounted Reward : 11.0
>>>

```

EXP 3 : Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation.

CODE :

```
gamma = float(input("Enter Discount Factor (0 to 1): "))
states = int(input("Enter Number of States: "))
state_value = []
reward = []
print("\nEnter Reward for Each State")
for i in range(states):
    r = float(input("Reward for State " + str(i) + ": "))
    reward.append(r)
    state_value.append(0)
iterations = int(input("\nEnter Number of Bellman Iterations: "))
for k in range(iterations):
    new_value = state_value.copy()
    for i in range(states):
        if i == states - 1:
            next_state = i
        else:
            next_state = i + 1
        new_value[i] = reward[i] + gamma * state_value[next_state]
    state_value = new_value
print("\nState Value Function")
for i in range(states):
    print("V(State", i, ") =", round(state_value[i], 2))
print("\nAction Value Function")
for i in range(states):
    if i == states - 1:
        next_state = i
    else:
```

```

    next_state = i + 1

    q = reward[i] + gamma * state_value[next_state]

    print("Q(State", i, ", Move) =", round(q,2))

print("\nPolicy")

for i in range(states - 1):

    print("State", i, "-> Move to State", i + 1)

print("State", states - 1, "-> Goal")

print("\nBellman Equation Used")

print("V(s) = R(s) +  $\gamma \times V(s')$ ")

```

OUTPUT :

```

=====
Enter Discount Factor (0 to 1): 1
Enter Number of States: 3

Enter Reward for Each State
Reward for State 0: 1
Reward for State 1: 4
Reward for State 2: 5

Enter Number of Bellman Iterations: 3

State Value Function
V(State 0 ) = 10.0
V(State 1 ) = 14.0
V(State 2 ) = 15.0

Action Value Function
Q(State 0 , Move) = 15.0
Q(State 1 , Move) = 19.0
Q(State 2 , Move) = 20.0

Policy
State 0 -> Move to State 1
State 1 -> Move to State 2
State 2 -> Goal

Bellman Equation Used
V(s) = R(s) +  $\gamma \times V(s')$ 
>>> |

```

EXP 4 : Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency.

CODE :

```

import random

import math

actions = int(input("Enter Number of Actions: "))

episodes = int(input("Enter Number of Episodes: "))

values = []

print("\nEnter Estimated Reward for Each Action")

for i in range(actions):

    v = float(input("Action " + str(i + 1) + ": "))

    values.append(v)

epsilon = float(input("\nEnter Epsilon Value (0 to 1): "))

```

```

temperature = float(input("Enter Softmax Temperature: "))
print("\nEpsilon-Greedy Strategy")
for i in range(episodes):
    if random.random() < epsilon:
        action = random.randint(0, actions - 1)
        method = "Explore"
    else:
        action = values.index(max(values))
        method = "Exploit"
    print("Episode", i + 1)
    print("Action:", action + 1)
    print("Method:", method)
    print()
print("\nSoftmax Strategy")
exp_values = []
for v in values:
    exp_values.append(math.exp(v / temperature))
total = sum(exp_values)
probability = []
for e in exp_values:
    probability.append(e / total)
for i in range(actions):
    print("Action", i + 1, "Probability =", round(probability[i],3))
best = probability.index(max(probability))
print("\nBest Action using Softmax :", best + 1)

```

OUTPUT :

```

===== RESTART: D:/MLA0302/CO3
Enter Number of Actions: 4
Enter Number of Episodes: 3

Enter Estimated Reward for Each Action
Action 1: 4
Action 2: 5
Action 3: 2
Action 4: 4

Enter Epsilon Value (0 to 1): 0
Enter Softmax Temperature: 12

Epsilon-Greedy Strategy
Episode 1
Action: 2
Method: Exploit

Episode 2
Action: 2
Method: Exploit

Episode 3
Action: 2
Method: Exploit

Softmax Strategy
Action 1 Probability = 0.254
Action 2 Probability = 0.276
Action 3 Probability = 0.215
Action 4 Probability = 0.254

Best Action using Softmax : 2
>>>

```

EXP 5 : Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies.

CODE :

```

episodes = int(input("Enter Number of Episodes: "))
total_reward = 0
print("\nReward Shaping Simulation\n")
for i in range(episodes):
    distance = int(input("Enter Distance to Goal: "))
    if distance == 0:
        reward = 20
    elif distance <= 2:
        reward = 10
    elif distance <= 5:
        reward = 5
    else:
        reward = -2
    total_reward += reward
    print("Episode:", i + 1)
    print("Reward:", reward)
    print("Cumulative Reward:", total_reward)
    print()

```

```
print("-----")
print("Training Completed")
print("-----")
print("Total Reward:", total_reward)
if total_reward > 30:
    print("Good Policy Learned")
elif total_reward > 10:
    print("Learning Improved")
else:
    print("Suboptimal Policy")
```

OUTPUT :

```
>>> =====
Enter Number of Episodes: 4

Reward Shaping Simulation

Enter Distance to Goal: 2
Episode: 1
Reward: 10
Cumulative Reward: 10
Enter Distance to Goal:
```


6) Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

CODE :

```
# Reinforcement Learning in Robotics - Challenge Analysis
# User Input Based Simulation

print("=== Reinforcement Learning in Real-World Robotics ===")
print("\nEnter values between 0 and 100.")
safety = float(input("Safety Level (%): "))
sample_efficiency = float(input("Sample Efficiency (%): "))
environment_variability = float(input("Environment Variability (%): "))

# Calculate Performance
performance = (
    (0.4 * safety) +
    (0.4 * sample_efficiency) +
    (0.2 * (100 - environment_variability))
)

print("\n----- RESULT -----")
print(f"Safety Level          : {safety}%")
print(f"Sample Efficiency      : {sample_efficiency}%")
print(f"Environment Variability : {environment_variability}%")
print(f"Overall Agent Performance : {performance:.2f}%")
print("\n----- ANALYSIS -----")

# Safety Analysis
if safety >= 80:
    print("Safety: Excellent - Robot can operate with minimal risk.")
elif safety >= 50:
    print("Safety: Moderate - Robot requires supervision.")
else:
    print("Safety: Poor - High probability of unsafe actions.")

# Sample Efficiency Analysis
if sample_efficiency >= 80:
```

```

    print("Sample Efficiency: Learns quickly with fewer training samples.")
elif sample_efficiency >= 50:
    print("Sample Efficiency: Moderate learning speed.")
else:
    print("Sample Efficiency: Requires many interactions for learning.")
# Environment Variability Analysis
if environment_variability <= 20:
    print("Environment Variability: Stable environment.")
elif environment_variability <= 60:
    print("Environment Variability: Moderate changes in environment.")
else:
    print("Environment Variability: Highly dynamic environment, learning becomes difficult.")
print("\n----- CHALLENGES -----")
print("1. Safety Issues")
print(" - Unsafe exploration may damage the robot.")
print(" - Human safety must always be ensured.")
print("\n2. Sample Efficiency")
print(" - Real robots cannot perform millions of trial-and-error actions.")
print(" - Learning should happen using fewer samples.")
print("\n3. Environment Variability")
print(" - Lighting, obstacles, weather, and object positions change.")
print(" - Policies trained in simulation may fail in the real world.")
print("\n4. High Training Cost")
print(" - Real-world training consumes time and energy.")
print("\n5. Hardware Wear and Tear")
print(" - Excessive exploration can damage motors and sensors.")
print("\n6. Sim-to-Real Gap")
print(" - Simulation environments differ from physical robots.")

print("\n----- FINAL EVALUATION -----")
if performance >= 80:
    print("Overall Result: Excellent RL performance for robotics.")

```

```

elif performance >= 60:

    print("Overall Result: Good performance with some improvements needed.")
elif performance >= 40:

    print("Overall Result: Average performance. Training should be improved.")
else:

    print("Overall Result: Poor performance. Focus on safety, efficient learning, and handling
environment changes.")

```

OUTPUT :

```

Enter values between 0 and 100.
Safety Level (%): 85
Sample Efficiency (%): 70
Environment Variability (%): 30

----- RESULT -----
Safety Level           : 85.0%
Sample Efficiency       : 70.0%
Environment Variability : 30.0%
Overall Agent Performance : 76.00%

----- ANALYSIS -----
Safety: Excellent - Robot can operate with minimal risk.
Sample Efficiency: Moderate learning speed.
Environment Variability: Moderate changes in environment.

----- CHALLENGES -----
1. Safety Issues
   - Unsafe exploration may damage the robot.
   - Human safety must always be ensured.

2. Sample Efficiency
   - Real robots cannot perform millions of trial-and-error actions.
   - Learning should happen using fewer samples.

3. Environment Variability
   - Lighting, obstacles, weather, and object positions change.
   - Policies trained in simulation may fail in the real world.

4. High Training Cost
   - Real-world training consumes time and energy.

5. Hardware Wear and Tear
   - Excessive exploration can damage motors and sensors.

6. Sim-to-Real Gap
   - Simulation environments differ from physical robots.

----- FINAL EVALUATION -----
Overall Result: Good performance with some improvements needed.

```

7) Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks.

CODE :

```
# Discount Factor (Gamma) Analysis in Reinforcement Learning
print("==== Discount Factor ( $\gamma$ ) Analysis ====")

# User Inputs
gamma = float(input("Enter discount factor ( $\gamma$ ) (0 to 1): "))
immediate_reward = float(input("Enter immediate reward: "))
future_reward = float(input("Enter future reward: "))
steps = int(input("Enter number of future steps: "))
episode = input("Is the task Episodic? (yes/no): ").strip().lower()

# Validation
if gamma < 0 or gamma > 1:
    print("\nError: Gamma must be between 0 and 1.")
    exit()

# Calculate discounted future reward
discounted_future = future_reward * (gamma ** steps)

# Total return
total_return = immediate_reward + discounted_future

print("\n===== RESULTS =====")
print(f"Discount Factor ( $\gamma$ )      : {gamma}")
print(f"Immediate Reward          : {immediate_reward}")
print(f"Future Reward             : {future_reward}")
print(f"Future Steps              : {steps}")
print(f"Discounted Future Reward: {discounted_future:.4f}")
print(f"Total Expected Return   : {total_return:.4f}")

# Decision-making analysis
print("\n----- Decision-Making Analysis -----")
if gamma < 0.3:
    print("• Very Low  $\gamma$ ")
    print("- Agent strongly prefers immediate rewards.")
```

```

    print("- Long-term rewards have very little influence.")
    print("- Suitable for highly reactive systems.")
elif gamma < 0.7:
    print("• Medium  $\gamma$ ")
    print("- Agent balances immediate and future rewards.")
    print("- Useful when both short-term and long-term gains matter.")
elif gamma < 0.95:
    print("• High  $\gamma$ ")
    print("- Agent values future rewards significantly.")
    print("- Encourages long-term planning and better strategies.")
else:
    print("• Very High  $\gamma$ ")
    print("- Agent almost equally values future and immediate rewards.")
    print("- Learning may become slower but focuses on optimal long-term returns.")
# Episodic task analysis
print("\n----- Episodic Task Evaluation -----")
if episode == "yes":
    if gamma >= 0.9:
        print("High  $\gamma$  is effective for episodic tasks.")
        print("The agent tries to maximize the total reward until the episode ends.")
    elif gamma >= 0.5:
        print("Moderate  $\gamma$  provides a balance between early and final rewards.")
    else:
        print("Low  $\gamma$  causes the agent to focus mainly on early rewards within the episode.")
else:
    print("For continuing (non-episodic) tasks:")
    if gamma >= 0.95:
        print("High  $\gamma$  helps maximize long-term cumulative rewards.")
    else:
        print("Lower  $\gamma$  makes the agent prioritize immediate gains.")

print("\n===== CONCLUSION =====")

```

```

if gamma < 0.5:
    print("Short-Term Decision Making:")
    print("- Immediate rewards dominate.")
    print("- Faster decisions.")
    print("- Less planning.")
elif gamma < 0.9:
    print("Balanced Decision Making:")
    print("- Both present and future rewards are considered.")
    print("- Good compromise for many RL problems.")
else:
    print("Long-Term Decision Making:")
    print("- Future rewards dominate.")
    print("- Better for strategic planning.")
    print("- Common in episodic tasks requiring long sequences of actions.")

```

OUTPUT:

```

=====
=== Discount Factor (γ) Analysis ===
Enter discount factor (γ) (0 to 1): 0.9
Enter immediate reward: 10
Enter future reward: 100
Enter number of future steps: 5
Is the task Episodic? (yes/no): yes

===== RESULTS =====
Discount Factor (γ)      : 0.9
Immediate Reward         : 10.0
Future Reward            : 100.0
Future Steps             : 5
Discounted Future Reward : 59.0490
Total Expected Return    : 69.0490

----- Decision-Making Analysis -----
• High γ
- Agent values future rewards significantly.
- Encourages long-term planning and better strategies.

----- Episodic Task Evaluation -----
High γ is effective for episodic tasks.
The agent tries to maximize the total reward until the episode ends.

===== CONCLUSION =====
Long-Term Decision Making:
- Future rewards dominate.
- Better for strategic planning.
- Common in episodic tasks requiring long sequences of actions.

```

8) Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations.

CODE :

```
import random
print("=" * 60)
print("MODEL-FREE RL vs MODEL-BASED RL COMPARISON")
print("=" * 60)
environment = input("Enter environment type (Static/Dynamic): ").strip().lower()
episodes = int(input("Enter number of episodes: "))
reward_high = float(input("Enter reward for successful action: "))
reward_low = float(input("Enter penalty for unsuccessful action: "))
print("\nSimulation Started...\n")
model_free_total = 0
model_based_total = 0
print("{:<10} {:<20} {:<20}".format("Episode", "Model-Free", "Model-Based"))
print("-" * 50)
for ep in range(1, episodes + 1):
    if environment == "dynamic":
        current_reward = random.choice([reward_high, reward_low])
    else:
        current_reward = reward_high
    # Model-Free Agent
    if random.random() < 0.6:
        mf_reward = current_reward
    else:
        mf_reward = reward_low
    # Model-Based Agent
    if environment == "dynamic":
        if random.random() < 0.85:
            mb_reward = current_reward
        else:
```

```

        mb_reward = reward_low
    else:
        if random.random() < 0.75:
            mb_reward = reward_high
        else:
            mb_reward = reward_low
    model_free_total += mf_reward
    model_based_total += mb_reward
    print("{:<10} {:<20} {:<20}".format(ep, mf_reward, mb_reward))
print("\n" + "=" * 60)
print("FINAL RESULTS")
print("=" * 60)
print("Total Reward (Model-Free RL):", model_free_total)
print("Total Reward (Model-Based RL):", model_based_total)
print()
if model_based_total > model_free_total:
    print("Model-Based RL performed better in this simulation.")
elif model_free_total > model_based_total:
    print("Model-Free RL performed better in this simulation.")
else:
    print("Both models performed equally.")
print("\nStrengths and Limitations")
print("-" * 60)
print("MODEL-FREE RL")
print("Strengths:")
print("1. Easy to implement.")
print("2. No environment model required.")
print("3. Suitable for unknown environments.")
print("Limitations:")
print("1. Requires more training data.")
print("2. Learns slowly.")
print("3. Adapts slowly to dynamic changes.")

```



```
print()
print("MODEL-BASED RL")
print("Strengths:")
print("1. Learns faster using environment model.")
print("2. Better planning and decision making.")
print("3. Adapts quickly in dynamic environments.")
print("Limitations:")
print("1. Needs an accurate environment model.")
print("2. More computationally expensive.")
print("3. Difficult to build for complex environments.")
print("\nEvaluation")
print("-" * 60)
if environment == "dynamic":
    print("Dynamic Environment:")
    print("Model-Based RL is generally more effective because it predicts")
    print("future states and replans when the environment changes.")
    print("Model-Free RL eventually adapts but usually requires more experience.")
else:
    print("Static Environment:")
    print("Model-Free RL performs well after sufficient training.")
    print("Model-Based RL still learns faster but offers less advantage.")
```

OUTPUT :

```
MODEL-FREE RL vs MODEL-BASED RL COMPARISON
=====
Enter environment type (Static/Dynamic): dynamic
Enter number of episodes: 10
Enter reward for successful action: 10
Enter penalty for unsuccessful action: -5

Simulation Started...

Episode    Model-Free    Model-Based
-----
1          -5.0         -5.0
2          -5.0         10.0
3          -5.0         10.0
4          -5.0         -5.0
5          10.0         10.0
6          -5.0         10.0
7          10.0         10.0
8          10.0         10.0
9          -5.0         -5.0
10         -5.0         -5.0

=====
FINAL RESULTS
=====
Total Reward (Model-Free RL): -5.0
Total Reward (Model-Based RL): 40.0

Model-Based RL performed better in this simulation.

Strengths and Limitations
-----
MODEL-FREE RL
Strengths:
1. Easy to implement.
2. No environment model required.
3. Suitable for unknown environments.
Limitations:
1. Requires more training data.
2. Learns slowly.
3. Adapts slowly to dynamic changes.

MODEL-BASED RL
Strengths:
1. Learns faster using environment model.
2. Better planning and decision making.
3. Adapts quickly in dynamic environments.
Limitations:
1. Needs an accurate environment model.
2. More computationally expensive.
3. Difficult to build for complex environments.

Evaluation
-----
Dynamic Environment:
Model-Based RL is generally more effective because it predicts
future states and replans when the environment changes.
Model-Free RL eventually adapts but usually requires more experience.
```

9) Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior.

CODE:

```
import numpy as np
import random

# -----
# User Input
# -----

n_states = int(input("Enter number of states: "))
n_actions = int(input("Enter number of actions: "))
episodes = int(input("Enter number of training episodes: "))
alpha = float(input("Enter learning rate (0-1): "))
gamma = float(input("Enter discount factor (0-1): "))
epsilon = float(input("Enter epsilon (0-1): "))

# -----
# Initialize Q-table
# -----

Q = np.zeros((n_states, n_actions))

# -----
# Reward Matrix
# -----

print("\nEnter Reward Matrix")
print(f'(Enter {n_states} rows, each with {n_states} space-separated values)')
print("Use -1 for invalid transitions if needed.")
R = np.zeros((n_states, n_states))

for i in range(n_states):
    R[i] = list(map(float, input(f'Row {i}: ').split()))

goal_state = int(input("\nEnter Goal State (0 to {}): ".format(n_states - 1)))

# -----
# Q-Learning Training
# -----
```

```

episode_rewards = []
for ep in range(epochs):
    state = random.randint(0, n_states - 1)
    total_reward = 0
    while state != goal_state:
        # Possible next states
        possible = [i for i in range(n_states) if R[state][i] != -1]
        if len(possible) == 0:
            break
        # Epsilon-Greedy
        if random.random() < epsilon:
            next_state = random.choice(possible)
        else:
            q_values = [Q[state][s % n_actions] for s in possible]
            next_state = possible[np.argmax(q_values)]
        reward = R[state][next_state]
        # Q-learning Update
        old_value = Q[state][next_state % n_actions]
        max_future = np.max(Q[next_state])
        new_value = old_value + alpha * (
            reward + gamma * max_future - old_value
        )
        Q[state][next_state % n_actions] = new_value
        total_reward += reward
        state = next_state
    episode_rewards.append(total_reward)
# -----
# Output
# -----
print("\nFinal Q-Table")
print(np.round(Q, 2))

```

```

print("\nBest Action for Each State")

for s in range(n_states):
    print(f"State {s} --> Action {np.argmax(Q[s])}")

print("\nConvergence (Episode Reward)")

step = max(1, episodes // 10)

for i in range(0, episodes, step):
    print(f"Episode {i+1}: Reward = {episode_rewards[i]}")

print("\nLast 10 Episode Rewards:")

print(episode_rewards[-10:])

```

OUTPUT :

```

----- RESTART: C:/
Enter number of states: 6
Enter number of actions: 6
Enter number of training episodes: 500
Enter learning rate (0-1): 0.8
Enter discount factor (0-1): 0.9
Enter epsilon (0-1): 0.2

Enter Reward Matrix
(Enter 6 rows, each with 6 space-separated values)
Use -1 for invalid transitions if needed.
Row 0: -1
Row 1: 100
Row 2: -1
Row 3: -1
Row 4: 100
Row 5: 100

Enter Goal State (0 to 5): 5

Final Q-Table
[[ 0. 0. 0. 0. 0. 0.]
 [ 100. 999.57 100. 100. 1000. 100.]
 [ 0. 0. 0. 0. 0. 0.]
 [ 0. 0. 0. 0. 0. 0.]
 [ 100. 1000. 100. 100. 1000. 100.]
 [ 0. 0. 0. 0. 0. 0.]]

Best Action for Each State
State 0 --> Action 0
State 1 --> Action 4
State 2 --> Action 0
State 3 --> Action 0
State 4 --> Action 1
State 5 --> Action 0

Convergence (Episode Reward)
Episode 1: Reward = 100.0
Episode 51: Reward = 0
Episode 101: Reward = 0
Episode 151: Reward = 700.0
Episode 201: Reward = 0
Episode 251: Reward = 0
Episode 301: Reward = 0
Episode 351: Reward = 0
Episode 401: Reward = 900.0
Episode 451: Reward = 0

```

10) Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications.

CODE:

```
import numpy as np
import random

# -----
# User Inputs
# -----

print("Reinforcement Learning: On-Policy vs Off-Policy")
print("1. SARSA (On-Policy)")
print("2. Q-Learning (Off-Policy)")
choice = int(input("Choose algorithm (1 or 2): "))
episodes = int(input("Enter number of episodes: "))
alpha = float(input("Enter learning rate (0-1): "))
gamma = float(input("Enter discount factor (0-1): "))
epsilon = float(input("Enter epsilon (0-1): "))

# -----
# Environment
# -----

n_states = 6
goal_state = 5
actions = [0, 1] # 0 = Left, 1 = Right
Q = np.zeros((n_states, len(actions)))

# -----
# Epsilon-Greedy Policy
# -----

def choose_action(state):
    if random.uniform(0, 1) < epsilon:
        return random.choice(actions)
    return np.argmax(Q[state])
```

```

# -----
# Training
# -----

for ep in range(epochs):

    state = 0

    action = choose_action(state)

    while state != goal_state:

        if action == 1:

            next_state = min(state + 1, goal_state)

        else:

            next_state = max(state - 1, 0)

        reward = 10 if next_state == goal_state else -1

        if choice == 1:

            # ----- SARSA (On-Policy) -----

            next_action = choose_action(next_state)

            Q[state, action] = Q[state, action] + alpha * (

                reward + gamma * Q[next_state, next_action] - Q[state, action]

            )

            state = next_state

            action = next_action

        elif choice == 2:

            # ----- Q-Learning (Off-Policy) -----

            best_next = np.max(Q[next_state])

            Q[state, action] = Q[state, action] + alpha * (

                reward + gamma * best_next - Q[state, action]

            )

            state = next_state

            action = choose_action(state)

# -----

```

```

# Results

# -----

print("\nTraining Completed!")

if choice == 1:

    print("Algorithm Used : SARSA (On-Policy)")

else:

    print("Algorithm Used : Q-Learning (Off-Policy)")


print("\nLearned Q-Table:")

print(np.round(Q, 2))

print("\nDerived Policy")

for s in range(n_states):

    if s == goal_state:

        print("State", s, "-> Goal")

    else:

        if np.argmax(Q[s]) == 1:

            print("State", s, "-> Move Right")

        else:

            print("State", s, "-> Move Left")

```

OUTPUT :

```

Reinforcement Learning: On-Policy vs Off-Policy -----
1. SARSA (On-Policy)
2. Q-Learning (Off-Policy)
Choose algorithm (1 or 2): 2
Enter number of episodes: 500
Enter learning rate (0-1): 0.1
Enter discount factor (0-1): 0.9
Enter epsilon (0-1): 0.1

Training Completed!
Algorithm Used : Q-Learning (Off-Policy)

Learned Q-Table:
[[ 1.4   3.12]
 [ 1.63  4.58]
 [ 2.96  6.2 ]
 [ 3.85  8.   ]
 [ 5.51 10.   ]
 [ 0.    0.   ]]

Derived Policy
State 0 -> Move Right
State 1 -> Move Right
State 2 -> Move Right
State 3 -> Move Right
State 4 -> Move Right
State 5 -> Goal

```